NETWORK SIMULATOR

Software Specification Report

Version 3.0 (Final Submission)

Document Date: May 25, 2026

Project Type: Console-based C++ full-stack network simulator

Prepared For: ITL351 Computer Networks Lab

Group Members:
Prakash Palsaniya (2023BITE055)
Kanhaiya Yadav (2023BITE076)

Contents

1 Purpose

This document defines the software specification for the Network Simulator project. The simulator demonstrates a complete, full-stack network architecture, from the Physical Layer up to the Application Layer, through a menu-driven C++ console application. The report formalizes the behavior implemented in the codebase into a proper submission-ready specification, updated to satisfy all minimum deliverables for Submissions 1, 2, and 3.

2 Scope

The system models educational network scenarios involving:

- **Physical Layer:** point-to-point and hub-based communication via links.
- **Data Link Layer:** switch-based forwarding and bridge behavior with MAC learning, Parity-based error control, and CSMA/CD access control modeling.
- **Network Layer:** IPv4 classless addressing (CIDR), static routing, RIP dynamic routing, and ARP for MAC resolution.
- **Transport Layer:** Go-Back-N sliding-window flow control and ephemeral port allocation.
- **Application Layer:** Custom application services (Echo and File Transfer) communicating over well-known ports.
- Collision and broadcast domain reporting.

The simulator is educational in nature, designed to clearly demonstrate protocol encapsulation, decapsulation, and forwarding behaviors. Functions of different layers run seamlessly in parallel.

3 Objectives

The project shall:

1. Provide a simple console workflow for running predefined topology demonstrations.
2. Show how data is encapsulated from Application down to Physical layer and decapsulated upon receipt.
3. Demonstrate the distinct forwarding rules of hubs, switches, bridges, and routers.
4. Illustrate protocol integration, including ARP for MAC resolution, RIP for dynamic routing, and Go-Back-N for reliable transport.
5. Report collision domain and broadcast domain counts for supported topologies.
6. Preserve a modular structure mapping OSI layers to distinct C++ modules.

4 Product Overview

4.1 System Type

The application is a standalone C++ console simulator with no GUI, no persistent storage, and no external service dependency during execution.

4.2 Intended Users

Students and instructors evaluating full-stack network topology and protocol demonstrations for ITL351 coursework.

4.3 Operating Context

The simulator is compiled and executed from the command line using a C++17-compatible compiler (e.g., g++ or MSVC via CMake).

5 Functional Specification

5.1 FR-1 Main Menu and Input Validation

The system shall display a main menu with options for Point-to-Point, Hub Star, Switch Star, Hybrid Topology, and a Full-Stack Routed Application Demo, along with an option to run all demos sequentially. It shall reject invalid input and continue prompting.

5.2 FR-2 Point-to-Point Demo

The system shall instantiate two hosts with fixed MAC/IP addresses, establish a direct two-way link, and send an application layer message from PC1 to PC2.

5.3 FR-3 Hub Topology Demo

The system shall instantiate five hosts connected to one hub, send a message from PC1 to PC3, and demonstrate the hub broadcasting the frame to all connected devices except the sender.

5.4 FR-4 Switch Topology Demo

The system shall instantiate five hosts connected to one switch. It will demonstrate dynamic MAC learning in the switch MAC table, selective forwarding to known destinations, flooding to unknown destinations, and CSMA/CD and Go-Back-N functionality.

5.5 FR-5 Hybrid Topology Demo

The system shall instantiate two separate hub-based stars connected through one switch. It will demonstrate intra-segment and inter-segment traffic flow and flooding behavior across the switch.

5.6 FR-6 Full Stack Routed Application Demo

The system shall instantiate a complex topology consisting of a client, a left switch, a bridge, two routers (R1, R2), a right switch, and a server. It shall:

- Assign classless IPv4 addresses and configure default gateways.
- Use ARP to discover next-hop MAC addresses.
- Perform RIP routing updates between routers to establish dynamic routes, supplementing static direct routes.
- Forward Application messages (Echo and File) across the network.
- Demonstrate full encapsulation and decapsulation at every node.

5.7 FR-7 Encapsulation and Delivery

The system shall construct nested data structures representing the OSI layers: Application Message → Transport Segment → Network Packet → Ethernet Frame. End devices shall only process packets destined for their IP, dropping others unless they are routers forwarding traffic.

5.8 FR-8 Logging

The system shall log major simulation events categorized by OSI layer (e.g., [PHYSICAL], [DATA-LINK], [NETWORK], [TRANSPORT], [APPLICATION]).

5.9 FR-9 Domain Reporting

The system shall report collision and broadcast domains. The Full Stack Routed Demo shall properly calculate and report 6 collision domains and 3 broadcast domains.

6 Protocol Specification

6.1 Data Link Protocols

CSMA/CD: Modeled as a medium availability check before frame transmission.
Parity Error Control: Counts '1' bits in the encoded payload and assigns a parity bit. Receiving devices discard frames failing parity validation.

6.2 Network Protocols

ARP (Address Resolution Protocol): Hosts and routers broadcast ARP requests to resolve next-hop IPs to MAC addresses. ARP replies are unicast back, updating an internal ARP cache.
RIP (Routing Information Protocol): Routers broadcast routing tables (networks and metrics). Receiving routers update their tables if a shorter path (lower metric) is found.
Longest Prefix Match: Router forwarding uses CIDR notation (e.g., 10.0.4.0/24) to find the most specific matching route for an IP.

6.3 Transport Protocols

Go-Back-N Flow Control: Maintains a sliding window (default size 4). Fragments payloads into segments, tracks sequence numbers, and processes ACKs.
Port Allocation: Assigns ephemeral ports (starting at 49152) to clients and uses well-known ports for application services.

6.4 Application Protocols

Echo Service: Listens on well-known port 7, echoes received payload.
File Service: Listens on well-known port 21, serves file contents based on filename payload.

7 Architecture and Design

7.1 Core Components

- **Models:** Structs for `EthernetFrame`, `NetworkPacket`, `TransportSegment`, and `ApplicationMessage`.
- **Logger:** Centralized logging system appending layer tags to console output.

7.2 Device Model

- **Host:** End device implementing Layers 2-7. Hosts applications, manages transport windows, resolves ARP, and connects to one physical link.
- **Hub:** Layer 1 repeater. Broadcasts incoming frames to all other ports.
- **Switch & Bridge:** Layer 2 devices. Maintain MAC tables, learn source MACs, forward known destinations, and flood unknowns.
- **Router:** Layer 3 device. Decrements TTL, resolves routes via longest prefix match, re-encapsulates frames with new MAC addresses, and participates in RIP.

8 Non-Functional Specification

- **Modularity:** The system separates headers and source files by concern (`app`, `core`, `devices`, `physical`, `protocols`, `simulator`, `analysis`).
- **Portability:** Standard C++17.

9 Assumptions, Constraints, and Limitations

- MAC and IP addresses are manually assigned; DHCP is not simulated.
- No physical transmission delay or timer-based retransmission behavior is modeled (simulation is synchronous).
- Switch MAC tables and ARP caches do not age or evict entries.

10 Build and Execution Specification

Compile from the project root using CMake, or manually using g++:

```
g++ -std=c++17 -Iinclude src/main.cpp src/core/*.cpp src/devices/*.cpp \
    src/protocols/*.cpp src/simulator/*.cpp src/app/*.cpp src/analysis/*.cpp \
    src/physical/*.cpp -o network_sim.exe
```

11 Citations and References

The implementations of shortest-path logic (RIP) and application services (Echo, File) are entirely custom-built for this simulator. No external open-source algorithms, codebases, or third-party applications (e.g., Telnet, FTP clients) were imported. Therefore, no external citations are required.

12 Conclusion

The Network Simulator satisfies its role as a full-stack educational simulator. It demonstrates parallel layer execution, precise encapsulation logic, dynamic routing, and modular design, fulfilling all requirements for Submissions 1, 2, and 3.